THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 01 OF 01

System Design #10: Change Data Capture (CDC) Pipeline

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

01

YOUR STUDY COMPANION

A beginner-friendly, decision-by-decision walkthrough Scenario: A financial-services transaction application **Example implementation:** PostgreSQL + Debezium + Kafka **Purpose:** Solution Architect interview preparation. This is a generic design, not a claim about American Express internal systems.

NAVIGATION

Inside this guide

01 1. The problem: one database, several teams that need its data

02 2. Start with the simplest design: let every consumer query the database

03 3. How the database knows a change happened

04 4. The first surprise: what about rows that already exist?

05 5. One change stream, but three independent destinations

06 6. Decide what an event should look like

07 7. Build the first real downstream copy

08 8. What happens when something fails?

09 9. Protect the data and keep its meaning stable

10 10. Size the system before choosing capacity

11 11. Final design, built one decision at a time

12 12. Operations: what must be visible in production?

13 13. When would we choose another design?

14 14. The short interview explanation

15 Official references

CHAPTER

1. The problem: one database, several teams that need its data

Imagine a customer buys something for **$25.00**. The payment application stores a record in a PostgreSQL database:

transaction_id	amount	status	updated_at
txn_1042	$25.00	PENDING	10:00:00

A few seconds later, the payment decision arrives and the application changes the row:

transaction_id	amount	status	updated_at
txn_1042	$25.00	APPROVED	10:00:03

The **payment application** is the source of truth. But three other systems also need this change:

1. A **customer-support dashboard** should show `APPROVED` instead of `PENDING`.
2. An **analytics warehouse** should analyze transaction-status changes over time.
3. A **search index** should let support staff find the latest transaction quickly.

Today, these systems keep asking the payment database for updates. As more consumers arrive, the primary transaction database gets extra queries, and each team has to build its own update-detection logic.

Our design problem: How do we send committed database changes to several independent systems, with low delay, without putting the critical payment database at unnecessary risk?

What CDC means: *Change Data Capture* is a way to observe changes in stored data (such as row inserts, updates, and deletes) and send those changes elsewhere. CDC copies information about changes; it does **not** decide whether a payment should be approved.

Requirements we will assume

These are **illustrative interview assumptions**, not product guarantees:

- Detect committed `INSERT`, `UPDATE`, and `DELETE` operations on approved transaction tables.
- New changes should usually reach downstream systems in seconds; use **10 seconds for 99%** of healthy-path updates as an example freshness target.
- Each consumer may fail or become slow without blocking payment processing.
- A new consumer must be able to load existing records, not just future changes.
- A consumer must be able to recover from a temporary outage, even if it processes some changes twice.
- Restrict access to sensitive information; avoid distributing cardholder data unnecessarily.

Outside scope: This is not a payment authorization engine, a double-entry ledger, or a synchronous source of financial truth. The downstream copies are allowed to be temporarily behind.

CHAPTER

2. Start with the simplest design: let every consumer query the database



```
                          |
               +----------+----------+
               |          |          |
               v          v          v
        Support service  Analytics job  Search updater
        (polls DB)      (polls DB)     (polls DB)
```

How it works: Every service periodically runs a query such as:

```
SELECT transaction_id, amount, status, updated_at
FROM transactions
WHERE updated_at > :last_seen_time;
```

New concept — polling: Checking repeatedly to see whether something changed. It is easy to build, but the database does work even when nothing changed.

Decision 1 — How should we detect changes?

- **Choose: Log-based CDC** rather than repeated database polling.
- **Why:** We want to observe committed changes continuously and avoid making every consumer scan the primary database. Log-based capture can also represent deletes explicitly.
- **Rejected alternative:** Timestamp polling is simpler and can be enough for a small, low-risk application. Here it is a poor default: it adds read traffic, requires careful high-watermark logic, may miss intermediate updates, and does not detect hard deletes without extra design.

A subtle point: A correctly engineered polling process can be reliable under restricted assumptions. Polling is not inherently broken; it just becomes more complex when we require multiple consumers, deletes, and good recovery behavior.

Our revised shape is:

```
Payment DB --> One change-capture service --> Other systems
```

We have solved *how to notice a change*, but not yet *how to read the database's log safely*.

CHAPTER

3. How the database knows a change happened

PostgreSQL uses a **write-ahead log (WAL)** to record changes needed for recovery. When the application commits a database transaction, PostgreSQL has already recorded the information required for that commit. PostgreSQL **logical decoding** can translate relevant WAL information into a stream of logical row changes. [1, 2]

Think of the layers this way:

```
Application: UPDATE transactions SET status='APPROVED' ...
                     |
                     v
          PostgreSQL commits write
                     |
                     v
              WAL records
                     |
                     v
```

```
           Logical decoding output
                       |
                       v
        'txn_1042 changed to APPROVED'
```

New concept — a committed change: A transaction's writes are treated as complete only when the database commits them. We do not want to publish a change from a transaction that later rolls back.

New concept — logical replication slot: PostgreSQL can maintain a position for a logical change stream. This helps the CDC reader continue from where it left off. An inactive or lagging slot can cause PostgreSQL to retain WAL files and consume disk space. [2]

Decision 2 — Read the database log or add application triggers?

- **Choose:** PostgreSQL logical decoding of the WAL.
- **Why:** We can capture committed row changes without editing every business write path. It centralizes the change-detection mechanism.
- **Rejected alternative:** Database triggers could write change rows to an audit table, but they add logic to transactions, require careful maintenance, and change source-database write behavior. Triggers can still be valid when log-based CDC is unavailable.

Decision 3 — Build a custom WAL reader or use a CDC connector?

- **Choose: Debezium's PostgreSQL connector**, hosted in Kafka Connect for this example. [1]
- **Why:** Debezium already implements PostgreSQL CDC, initial snapshots, change-event envelopes, and ongoing streaming. We would rather operate a supported connector than invent database-specific replication logic.
- **Rejected alternative:** A custom WAL reader would give us control, but we would own protocol handling, schema changes, snapshots, offsets, and recovery. That is unnecessary complexity for this case.

New concept — Kafka Connect: A framework that runs data connectors and manages their tasks and configurations. **Debezium** is the component that reads PostgreSQL changes; **Kafka Connect** is the runtime hosting it. A single Debezium PostgreSQL connector uses one capture task, so adding Kafka Connect workers does not automatically make one source connector read the WAL in parallel. [1]

CHAPTER

4. The first surprise: what about rows that already exist?

Suppose the transactions table has **20 million rows** before we install CDC. Watching future updates is not enough; a new support database also needs today's existing transactions.

Our first idea might be:

```
Step A: SELECT * FROM transactions
Step B: Start reading new WAL changes
```

The gap is dangerous. If `txn_1042` changes between A and B, the new system may retain the old status forever.

New concept — snapshot: A point-in-time view of existing rows, used to create a starting state for a new downstream system.

Decision 4 — How do we bootstrap existing data without missing the handoff?

- **Choose:** Debezium's coordinated **initial snapshot plus continuous streaming**. It establishes a consistent starting view and streams later changes from an appropriate log position. [1]
- **Why:** A consumer needs both *the rows that already existed* and *changes that happen while the initial copy is in progress*.

- **Rejected alternative:** An unrelated bulk `SELECT *` followed by starting CDC later has a race window in which changes can disappear from the new copy.

```
Time ------------------------------------------------>
      Snapshot starts       Snapshot ends
            |====================|
Source:        writes may continue throughout
CDC:      [read current rows] -> [stream from saved log position]
Result:   existing state + subsequent changes, no handoff gap
```

Trade-off: A large snapshot can create read and I/O pressure on PostgreSQL. Throttle it, scope it to approved tables, and plan for source capacity. Debezium also supports incremental-snapshot approaches for appropriate backfill needs. [1]

CHAPTER

5. One change stream, but three independent destinations

We can now read changes. Should Debezium send each event directly to support, search, and analytics?

That seems simple, until the warehouse is unavailable. Then the capture service needs separate retry queues, progress tracking, and failure handling for every destination.

Decision 5 — Send directly to consumers or place a broker in the middle?

- **Choose: Apache Kafka** as a durable, replayable event stream between capture and consumers. [3]
- **Why:** Kafka stores events for a configured period. Each downstream team can read independently, stop, restart, and catch up within the retention window. One slow consumer does not need to block the others.
- **Rejected alternative:** Direct connector-to-warehouse integration is simpler when there is only one destination and no need to replay or fan out data. We reject it here because we have several independent destinations.

New concept — broker: A system that stores and distributes messages between producers and consumers. Kafka is an event-streaming platform that uses **topics** to organize records.

```
PostgreSQL --> Debezium --> Kafka topic
                                |
                 +--------------+--------------+
                 |             |              |
                 v             v              v
             Support        Analytics       Search
             consumer       consumer        consumer
```

New concept — consumer group: Consumers using the same group share processing of a topic's partitions. Independent applications use **different groups**, so support, analytics, and search each receive the complete stream. [3]

Important boundary: Kafka buffers work after a change is committed. A prolonged connector or Kafka outage can still build up WAL in PostgreSQL. Kafka does not eliminate source-side CDC risks.

CHAPTER

6. Decide what an event should look like

Let's capture the approval of `txn_1042`.

This is a **simplified teaching example**, not the exact complete Debezium wire format:

```
{
  "key": { "transaction_id": "txn_1042" },
  "value": {
    "op": "u",
    "before": { "status": "PENDING" },
    "after": {
      "transaction_id": "txn_1042",
      "amount_minor": 2500,
      "currency": "USD",
      "status": "APPROVED"
    },
    "source": { "table": "transactions" }
  }
}
```

Reading it: `op = u` means an update. `before` is information about the earlier row; `after` is the newer state. Actual field availability, especially `before`, depends on PostgreSQL replica identity and connector settings. `amount_minor = 2500` represents $25.00 if the currency uses two decimal places. [1]

Debezium's common operation labels include `r` for snapshot read, `c` for create, `u` for update, and `d` for delete. [1]

Decision 6 — Raw database rows or meaningful business events?

- **Choose:** For this problem, publish **row-change events** describing what changed in the approved source tables.
- **Why:** Search indexing and read-model synchronization need the latest database state; a row-change record directly serves that purpose.
- **Rejected alternative:** Publishing only business events such as `PaymentAuthorized` would require the payment application to define and reliably write those events. That is better for business workflows but is a different contract.

Critical difference: `status` changing to `APPROVED` is a database fact. `PaymentAuthorized` is a *business statement* whose meaning must be explicitly owned. CDC does not magically turn one into the other.

Decision 7 — How should we organize and order Kafka events?

- **Choose:** Initially use one governed topic per captured table and the stable `transaction_id` as the record key.
- **Why:** Updates to the same transaction usually map to the same Kafka partition. Kafka preserves order **within a partition**, which makes a latest-state consumer easier to implement. [3]
- **Rejected alternative:** Random record keys may distribute traffic evenly but can send two updates for the same transaction to different partitions, with no per-transaction processing order.

New concept — partition: A topic is split into ordered logs called partitions. Partitioning allows consumer parallelism, but **Kafka does not provide one global order across every partition or topic**. Changing partition counts can also change key routing, so it needs planning.

For deletes, consumers must interpret Debezium's delete event and any accompanying **tombstone** correctly. A Kafka tombstone (a key with a null value) is useful for log compaction; it is not a substitute for defining sink-deletion behavior. [1, 3]

CHAPTER

7. Build the first real downstream copy

The support dashboard needs the **latest state** of each transaction. Its consumer reads Kafka events and writes to a small support-facing database.

```
Kafka event for txn_1042
          |
          v
Support projector (consumer)
          |
          v
UPSERT transaction txn_1042 = APPROVED
          |
          v
Support read-model database
```

New concept — projection/read model: A database or index built for a particular query need. It is a copy derived from the authoritative source. **Upsert** means insert if absent, otherwise update.

Decision 8 — Share one downstream database or create independent projections?

- **Choose:** Give support, search, and analytics separate consumers and storage optimized for their own queries.
- **Why:** Support needs fast lookups, search needs an index, and analytics needs historical and aggregate queries. They can scale and recover separately.
- **Rejected alternative:** One shared downstream database would reduce the number of stores but couple teams, workloads, access controls, and deployment schedules.

Example flows: Support updates its latest-state SQL table; search updates a search index; analytics appends approved changes to an object store or warehouse. None of these sinks replaces the payment database.

Decision 9 — How do we survive receiving the same event twice?

Imagine this sequence:

1. The support consumer reads the `APPROVED` event.
2. It updates its support database successfully.
3. It crashes **before saving its Kafka progress**.
4. On restart it receives the same event again.

New concept — offset: A Kafka partition position representing how far a consumer has read. An offset is like a bookmark; a crash can happen between writing the sink and moving the bookmark.

- **Choose:** Design for **at-least-once processing** with an **idempotent sink**. Record progress only after a durable sink write, or store the sink update and processed position together when the sink supports a single transaction.
- **Why:** At-least-once means a record may be retried. An idempotent operation can be repeated without changing the final result beyond applying it once.
- **Rejected alternative:** Assume universal end-to-end exactly-once processing. Broker/connector features do not automatically make arbitrary external database writes or financial side effects exactly once.

For the latest-state support table, applying `status = APPROVED` twice is normally harmless. For a *money transfer*, doing the transfer twice would be disastrous; use durable event IDs, uniqueness constraints, and a properly designed financial workflow instead.

Ordering safeguard: An older replayed event must not overwrite a newer state. Preserve each key's processing order and, when offsets/state are stored separately or parallel processing is used, use a trustworthy version or source-position strategy. Do not rely on wall-clock timestamps alone.

CHAPTER

8. What happens when something fails?

A good system design must explain **where data waits, what retries, and who is at risk**.

Failure A — The analytics consumer stops

Kafka still holds events during the configured retention period. The support and search consumer groups keep running. The analytics consumer later reads from its last safe offset.

Choose: Retry and catch up from Kafka; **why:** consumers are independent; **rejected:** pause all consumers and block the payment database.

Failure B — Debezium or Kafka becomes unavailable

PostgreSQL continues to process normal database transactions as long as its own resources remain healthy, but the replication slot may retain WAL because the CDC reader is behind. If WAL grows until the source runs out of disk, the **primary system is endangered**. [2]

Choose: Monitor replication slot lag, retained WAL bytes, source disk space, connector state, and Kafka availability; recover the capture path before capacity runs out. **Rejected:** treat Kafka outages as a downstream-only issue.

Failure C — PostgreSQL fails over to another primary

The new primary must support continuity of the logical replication slot or a documented recovery/re-snapshot procedure. Slot failover behavior depends on PostgreSQL version, deployment, and configuration. [1, 2]

Choose: Validate failover support and rehearse the recovery plan. **Rejected:** assume every managed PostgreSQL failover transparently preserves the CDC stream.

Failure D — A malformed event or schema change breaks a consumer

Choose: Fail visibly, alert, place a rejected event in a controlled quarantine/dead-letter process when suitable, fix the consumer or event, then replay in order. **Rejected:** silently skip it and claim the data is consistent.

New concept — dead-letter queue (DLQ): A separate place to investigate events that cannot be processed normally. A DLQ is not proof of correctness; it needs ownership and a repair procedure.

Decision 10 — What is our recovery contract?

- **Choose:** Retain Kafka events for a tested recovery window and keep a controlled snapshot/rebuild path if older history is unavailable.
- **Why:** A consumer behind Kafka's retention limit cannot read deleted historical records from Kafka. Separate durable archives may extend recoverability.
- **Rejected alternative:** Assume Kafka keeps all events forever or that an old offset can always be resumed.

Two retention budgets: Kafka retention controls *how long consumers can replay*. PostgreSQL WAL retention controls *how much source log the connector has not yet consumed*. They are different and both need monitoring.

CHAPTER

9. Protect the data and keep its meaning stable

A CDC system can accidentally copy private database columns to many teams. That increases security and compliance scope.

Decision 11 — What data and schemas may appear in the stream?

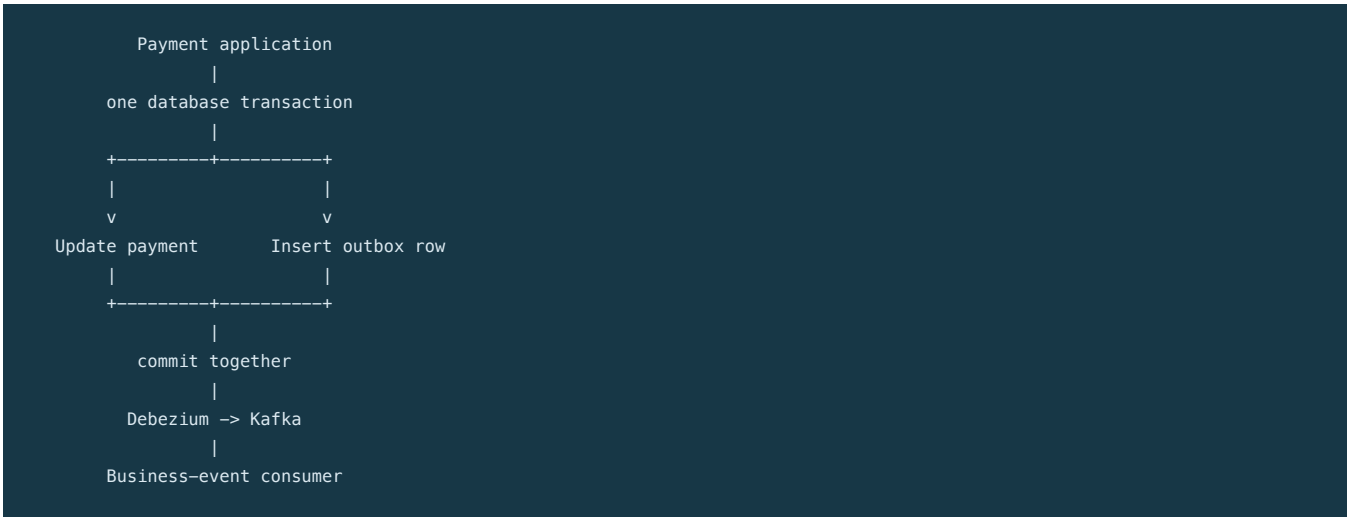
- **Choose:** An explicit allowlist of source tables and fields, versioned event contracts, topic permissions, transport encryption, and approved consumers.
- **Why:** Downstream systems need predictable event shapes and should receive only the information they actually need.
- **Rejected alternative:** Publish every table and column as unrestricted JSON. This makes schema changes surprising and sensitive data hard to control.

New concept — schema evolution: A table can gain a new column or change a data type over time. Consumers need a tested policy for old and new event shapes. A schema registry plus a chosen format can help enforce compatibility, but a registry is optional if equivalent governance exists.

For a financial-services example, **do not copy full card numbers or security codes** into general Kafka topics. Use minimization and tokenization where appropriate. Topic retention, replay, logging, and backups affect where protected data might live.

Decision 12 — Should every business action be driven by CDC rows?

- **Choose:** Keep CDC for read-model synchronization. If a downstream service must react to a business event such as `PaymentAuthorized`, use a **transactional outbox** when appropriate. [4]
- **Why:** The payment service can write its business state and an `outbox_event` row in the **same database transaction**. CDC then delivers the outbox row. This avoids a fragile database-then-Kafka dual write and gives the event a controlled business meaning and stable event ID.
- **Rejected alternative:** Trigger non-idempotent business actions directly from arbitrary changes to a `transactions.status` column.



What the outbox does not solve by itself: The consumer must still handle retries and duplicates. The outbox makes writing the business state and the event record atomic; it does not make every downstream side effect automatically exactly once.

CHAPTER

10. Size the system before choosing capacity

Let's size a **small illustrative starting point**. These are assumptions for learning, **not real American Express traffic or vendor limits**.

Item	Example assumption	Result
Average changes	100 events/second	—

Peak changes	1,000 events/second	10x burst
Average Kafka event size	1 KB	—
Daily event count	100 × 86,400	8.64 million
Data per day, before overhead	8.64 million × 1 KB	~8.64 GB
Seven-day logical retention	8.64 GB × 7	~60.5 GB
Three Kafka copies, before overhead	60.5 GB × 3	~181.4 GB

Actual capacity needs compression, topic metadata, safety headroom, growth, partition count, snapshot demand, and consumer catch-up planning.

Decision 13 — Where should we scale first?

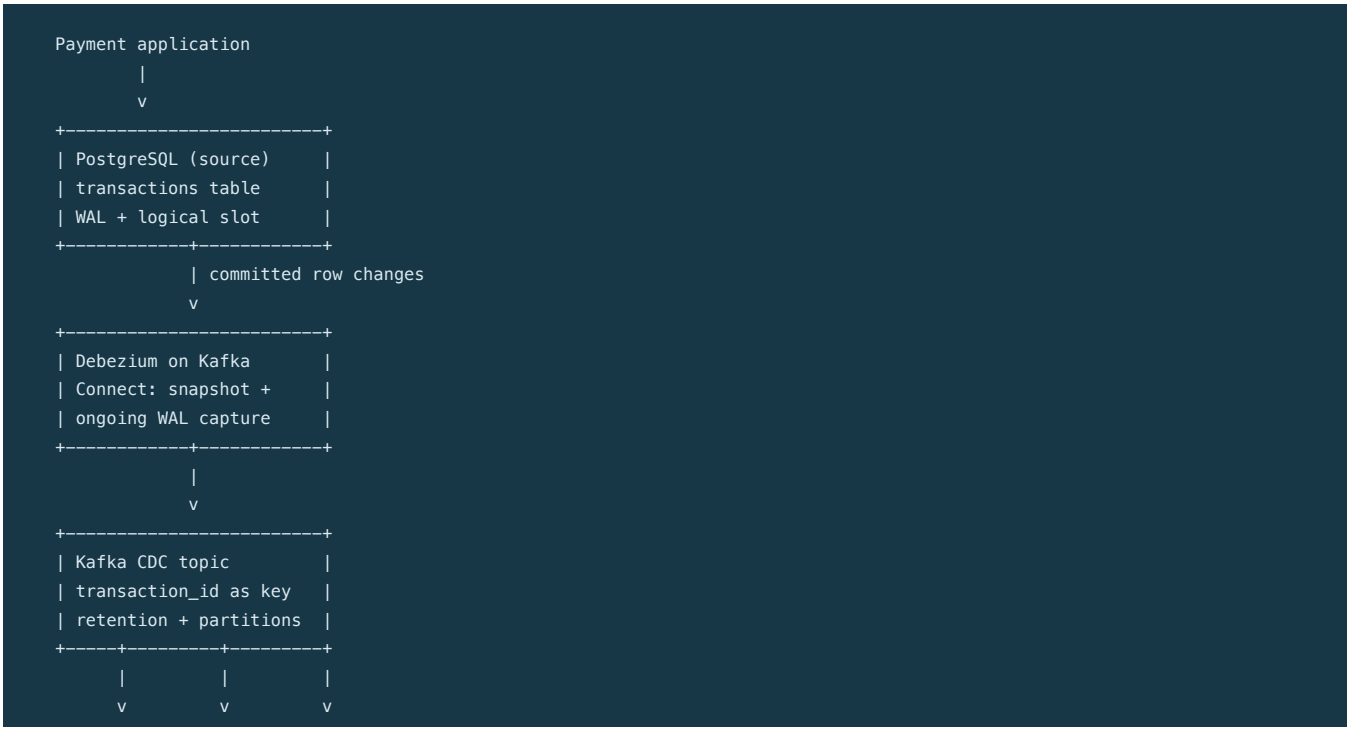
- **Choose:** Start with measured source CDC throughput, Kafka broker capacity, and independent consumer scaling. Partition topics for expected consumer concurrency.
- **Why:** Adding more consumer instances helps only when there are enough partitions to divide the work. One PostgreSQL Debezium connector has a single capture task, so extra Kafka Connect workers do not multiply its capture throughput. [1, 3]
- **Rejected alternative:** Add dozens of consumers or Connect workers before measuring the actual bottleneck.

For initial snapshots, protect the payment database with bounded load. For continued streaming, verify the connector can keep up with the sustained WAL change rate.

CHAPTER

11. Final design, built one decision at a time

We can now assemble the final design. Each box exists for a specific reason.



```
  Support     Analytics   Search
  consumer    consumer    consumer
      |           |          |
      v           v          v
  Read model  Warehouse   Search index

Across all parts: permissions, encryption,
schema management, lag alerts, replay, reconciliation
```

Walk through `txn_1042` from start to finish

1. The payment application commits `txn_1042: PENDING -> APPROVED` in PostgreSQL.
2. The change is available through WAL logical decoding; the replication slot tracks progress.
3. Debezium turns the committed update into a structured change event.
4. Kafka stores that event in the partition selected by `transaction_id`.
5. The support consumer updates the support read model to `APPROVED`.
6. The search consumer updates its index; the analytics consumer stores the change for reports.
7. If analytics is down, it catches up later from Kafka within the configured retention window.
8. Monitoring checks whether source-to-sink freshness and CDC source safety are within limits.

Consistency we promise: The payment database is authoritative. Downstream views are **eventually consistent**, meaning they can briefly show older values. Ordering is per Kafka partition/key, not global across every system. Sink writes are designed for at-least-once delivery and safe replay.

CHAPTER

12. Operations: what must be visible in production?

Signal	Simple question it answers
End-to-end freshness	How many seconds until a committed change is visible in the sink?
Debezium connector/task state	Is change capture running?
PostgreSQL replication-slot/WAL lag	Is the source retaining too much WAL?
PostgreSQL free disk space	Could CDC lag threaten the source database?
Kafka topic/broker health	Can we reliably accept and retain changes?
Consumer lag	How far behind is support, search, or analytics?
Failed event count / DLQ	Are some events being rejected?
Source-to-sink reconciliation	Did a downstream copy silently diverge?

Choose: Monitor both *business freshness* and *infrastructure safety*. **Why:** Consumer lag can harm user experience, while growing retained WAL can threaten source availability. **Rejected:** monitor only whether the Kafka cluster is online.

Recovery runbooks to prepare: connector restart, Kafka outage, PostgreSQL primary failover, consumer replay, re-snapshot after retention loss, schema rollback, and data reconciliation. Validate these using real failure tests, not only a diagram.

CHAPTER

13. When would we choose another design?

Requirement	Better-fit approach	Why not our default CDC + Kafka design?
Small table, hourly updates, no deletes	Timestamp polling	Simpler, potentially enough for modest requirements
Only one warehouse needs nightly data	Batch ETL or a managed direct sink	Kafka adds unnecessary components
Another PostgreSQL database needs a replica	Native logical replication	A full multi-consumer integration platform may be unnecessary
Business consumers require `PaymentAuthorized`	Transactional outbox + CDC	A database row update lacks a complete business contract
Need immediate read-after-write correctness	Query the authoritative service/database	CDC is asynchronous, so the projection may lag
Need a legally complete long-term audit record	Governed append-only audit design	Compacted or time-limited Kafka topics are not a legal audit store

CHAPTER

14. The short interview explanation

> A payment database stores transaction changes, but support, search, and analytics need their own copies. I would keep PostgreSQL as the source of truth. Rather than have every service poll it, I would use Debezium to capture committed changes from PostgreSQL's WAL. An initial snapshot provides existing records, and streaming then provides new changes without a gap. I would send events to Kafka so each downstream consumer can process independently and replay after failures. I would key records by transaction ID for per-key ordering, and make sink updates idempotent because retries can duplicate events. The operational risks are CDC lag retaining source WAL, schema changes, and consumer drift, so I would monitor and test those explicitly. If the requirement is a true business event, I would use a transactional outbox instead of treating a row change as the event itself.

Key points to remember

- **CDC answers:** What committed database rows changed?
- **WAL answers:** Where can PostgreSQL expose these changes efficiently?
- **Debezium answers:** Who reads those changes and converts them into events?
- **Kafka answers:** How can many consumers receive, buffer, and replay events independently?
- **Snapshot answers:** How do we copy existing data before watching future changes?
- **Idempotency answers:** How do retries avoid damaging results?
- **Outbox answers:** How do we reliably publish *business events*, not just row mutations?
- **Operational safety answers:** How do we stop a stalled CDC reader from endangering the source database?

CHAPTER

Official references

The design's implementation details should be checked against the exact product versions deployed. These references are maintained by the relevant projects.

1. **Debezium, PostgreSQL connector:** https://debezium.io/documentation/reference/stable/connectors/postgresql.html — snapshots, events, connector operation, PostgreSQL integration.
2. **PostgreSQL, logical decoding concepts:** https://www.postgresql.org/docs/current/logicaldecoding-explanation.html — WAL decoding, replication slots, repeats after crash, source-log considerations.
3. **Apache Kafka, design documentation:** https://kafka.apache.org/42/design/design/ — topics, partitions, consumer groups, delivery and persistence concepts.
4. **Debezium, outbox event router:** https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html — outbox event pattern and transformation.

Final takeaway: Build a CDC pipeline by solving one real problem at a time: detect committed changes, bootstrap existing data, distribute updates, preserve per-key order, survive retries, protect sensitive data, and keep the source database safe.